ChipChat: Low-Latency Cascaded Conversational Agent in MLX

Tatiana Likhomanenko*, Richard He Bai*, Zijin Gu*, Zakaria Aldeneh*, Shiladitya Dutta*, Luke Carlson*, Han Tran*, Yizhe Zhang, Ruixiang Zhang, Huangjie Zheng, Navdeep Jaitly*
Apple

arXiv:2509.00078v2 [eess.AS] 19 Jul 2026

***Abstract*—The emergence of large language models (LLMs) has transformed spoken dialog systems, yet the optimal architecture for real-time on-device voice agents remains an open question. While end-to-end approaches promise theoretical advantages, cascaded systems (CSs) continue to outperform them in language understanding tasks, despite being constrained by sequential processing latency. In this work, we introduce ChipChat, a novel low-latency CS that overcomes traditional bottlenecks through architectural innovations and streaming optimizations. Our system integrates streaming (a) conversational speech recognition with mixture-of-experts, (b) state-action augmented LLM, (c) text-to-speech synthesis, (d) neural vocoder, and (e) speaker modeling. Implemented using MLX, ChipChat achieves sub-second response latency on a Mac Studio without dedicated GPUs, while preserving user privacy through complete on-device processing. Our work shows that strategically redesigned CSs can overcome their historical latency limitations, offering a promising path forward for practical voice-based AI agents.**

***Index Terms*—Conversational agents, streaming systems, cascaded systems, MLX.**

I. Introduction

The landscape of spoken dialogue systems has undergone a profound transformation with the advent of large language models (LLMs). Traditionally, these systems relied on cascaded architectures that combine isolated specialized components—automatic speech recognition (ASR), language understanding, planning, response generation, text-to-speech synthesis (TTS), etc.—into sequential processing pipelines [1]. Recently, LLMs are used as the "interface" between these cascaded components explicitly [2], [3]. However, researchers and industry practitioners are now pivoting toward a fundamentally different approach: integrating LLMs directly with speech processing capabilities to create comprehensive end-to-end (E2E) speech LLMs that can power conversational agents (CAs) through unified architectures [4]–[12].

While this shift toward E2E speech LLMs represents a promising direction, it's important to understand the fundamental limitations that drove researchers away from cascaded architectures. The sequential nature of processing in cascaded architectures creates barriers to implementing low-latency systems, as each component must wait for complete outputs from preceding stages, thus degrading user experience [13], [14]. Additionally, these systems lose important information (e.g., speech characteristics are discarded when transcribing user speech into text) and suffer from error accumulation, where mistakes in earlier components propagate and amplify through the pipeline [10], [14], [15]. These fundamental limitations explain why the field has embraced E2E approaches that integrate speech and text processing into unified LLM architectures, promising improved quality, lower latency, architectural simplicity, and reduced maintenance complexity.

TABLE I
Latency of ChipChat components and subsystems tested with Apple Silicon (Mac Studio, M2 Ultra, 192GB, 2023). Time: t (ms).

	Wait t for input	Inference t per output	Total t up to the point
🎙	0	0	0
Mel	10	1	11
ASR	160	13	165-175
LLM State	[pause]	$\sim$ 560	$\sim$ 560
LLM	0	16	$\sim$ 576
TTS	[5 words]	20	$\sim$ 880
Vocoder	25	13	$\sim$ 920
🔊	25	0.2	$\sim$ 920

However, recent research challenges the presumed superiority of and belief in E2E systems. Several studies [4], [10], [15], [16] demonstrate that cascaded systems (CSs) combining state-of-the-art components—such as advanced ASR models with powerful LLMs, or LLMs with a high-quality TTS—consistently outperform their E2E speech LLM counterparts in spoken language understanding and dialog tasks in terms of accuracy. Furthermore, the quality gap appears to widen in complex scenarios involving multi-turn dialogues and domain-specific knowledge, precisely the areas most critical for practical applications [17]. Lastly, CSs are more interpretable as we can track what errors each component makes.

The streaming capabilities of E2E systems, once considered their defining advantage, have also come under scrutiny. While theoretically low latency can be provided, many E2E models still exhibit chunking behaviors and processing delays that impact user experience due to non-streaming speech encoders (e.g., Whisper) [9], [10] or speech tokenizations [18]. This raises fundamental questions: Is it possible to design CSs that (a) maintain their quality advantages while achieving the responsiveness that users expect from conversational agents, and (b) run fully on-device to preserve user privacy?

In this paper, we introduce ChipChat, a novel low-latency cascaded system for CAs that directly addresses the discussed historical limitations. ChipChat is based on: (a) classic design

*Core contributors.

© 2025 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

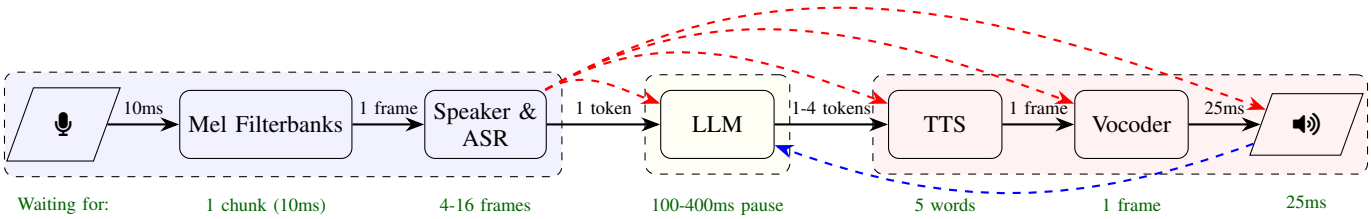

Fig. 1. ChipChat's streaming cascaded architecture. Red dashed lines show interruption signals sent by the ASR to downstream components. The blue dashed line represents the feedback mechanism when the audio player sends the feedback information back to the LLM to clear unvocalized generated responses. Text in green shows the input chunk that a process awaits before starting its generation.

of streaming ASR with mixture-of-experts (MoE) integration, (b) a state-action augmented LLM, (c) a low-latency streaming TTS, (d) a low-latency streaming neural vocoder, and (e) a real-time speaker modeling. Our approach preserves the performance benefits of specialized components while featuring a streaming design that enables progressive processing and generation across the entire pipeline. Leveraging MLX [19] for components' inference, we are able to achieve sub-second response latency for the entire system on Mac Studio, see Table I, making advanced CAs more accessible to users while preserving user privacy. These findings indicate that the future of CAs may not require abandoning cascaded approaches but rather rethinking them to address their inherent limitations.

II. ChipChat Design

ChipChat is a distributed system implemented in Python and designed to maximize both memory efficiency and inference speed while minimizing latency. We utilize RabbitMQ [20] for inter-process message passing, allowing each component to operate independently while maintaining a cohesive pipeline. This design allows each component's output to be streamed immediately upon generation, thus reducing latency. The system leverages specialized models for ASR, TTS generation, speaker representation, and dialog LLM augmented with state-action. An overview of ChipChat is given in Figure 1.

A. Main Components

Microphone Input: We implement browser-based microphone streaming with near-zero latency audio capture. We use the built-in noise cancellation (or voice isolation) capabilities from either the browser or the device hardware and process audio in 10ms chunks to be aligned with the feature computation process while reducing the number of sent messages.

Mel Filterbanks are computed with 10ms hop distance and 25ms window size and are normalized (zero mean and unit std). Due to normalization we perform running mean and std computation to preserve the streaming property.

ASR Model employs a transformer encoder architecture enhanced with MoE in fully connected layers from [21]. The model is trained with Connectionist Temporal Classification [22] loss and causal masking to enable streaming inference. The model uses 4-frame stacking, 8k word-piece tokenization and 4 experts resulting in $\sim$ 650M total parameters. To optimize inference speed, we (a) use batched inference (16 input frames found to be optimal); (b) implement key-value caching; (c) introduce a cache resetting mechanism as the training sequence length is less than 30s. The ASR component also functions as a voice activity detector (VAD) to identify user pauses and turn switches. Non-blank, non-repetitive tokens are immediately streamed to the LLM process, where they are encoded right away.

Speaker Model is based on the transformer from [23] and provides speaker diarization. We re-train a supervised variant of the model with the same feature extraction as ASR and integrate it directly in the ASR process to leverage its VAD. At the beginning of the conversation we wait for 3s audio to run the speaker model for the first time and enroll the extracted embedding vector. Subsequently we use a 1.5s sliding window, reducing the latency to 1.5s, and call the speaker model only if the current segment contains more than 20% of speech to avoid model inference for silence segments.

LLM: We employ a dialog LLM enhanced with state-action from [24]: first, the model infers the user's motivation and emotion, then it predicts the agent's motivation and emotion, and finally it predicts the agent's response. The model follows an 8x7B Mixtral model (8 experts, 45B total parameters) [25]. To optimize inference, (a) we pre-encode the conversation prompt in advance before the conversation begins; (b) ASR streams tokens as soon as they are generated, thus the LLM can encode them right away; (c) during LLM turns we pre-encode prompts for the user/agent states in a batch regime without generation; (d) we use the MLX-LM [26] key-value rotating cache. After the LLM generates user and agent states, every generated response token is streamed to the TTS process.

TTS model is based on the streaming architecture SpeakStream from [27]. It uses (a) the streaming speech tokenization via discretization of Mel filterbanks (dMel) [28]; (b) specially designed interleaving between written n-gram words and spoken n-gram words. Such interleaving design enables low-latency generation without loss in the quality. The TTS employs a transformer decoder (300M parameters) that produces frames at 40Hz. To optimize inference, (a) we pre-encode audio prefix used for every sentence generation to better preserve consistency and style; (b) LLM streams tokens as soon as they are generated, thus TTS encodes them right away as they arrive; (c) we implement key-value caching and reset the cache as a new sentence starts. The system waits for 5 words from the LLM and starts generation afterwards with

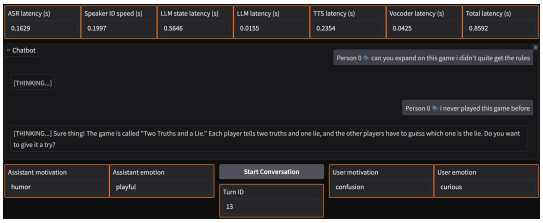

Fig. 2. ChipChat's Gradio viewer interface.

sending generated tokens immediately to the vocoder.

Vocoder: We use VocStream vocoder from [27]. It is a ParallelWaveGAN model with a fully causal convolutional architecture (13M parameters). VocStream consists of an upsampler (upsamples 40Hz input to 160Hz) and high-resolution vocoder (converts 160Hz input to the 24kHz wave). For each incoming frame, the vocoder generates the corresponding audio chunks that are immediately passed to the audio player.

Audio player introduces negligible latency, playing each audio chunk as soon as it becomes available from the vocoder.

Viewer: We also implement a process with Gradio [29] to showcase user and agent turns in the text form, user and agent motivation and emotion states predicted by LLM, etc. and to report performance of all components in real time (Figure 2).

B. Interruption Handling

Interruption management is a critical aspect of natural conversational systems that has been approached differently. Some systems, like Moshi [7], continue generation despite receiving user input, making interruptions difficult. Others terminate immediately, potentially creating abrupt transitions [12].

ChipChat implements a responsive interruption system that balances naturalness with responsiveness. Our architecture enables immediate detection of user speech through the ASR component, which continuously monitors the audio input. When a non-silence token is detected, the ASR process instantaneously signals all downstream components to halt generation (see Figure 1), creating a responsive system that yields the conversational floor when the user begins speaking.

This interruption capability is enhanced by our interruption feedback mechanism: when generation terminates due to user interruption, the audio player sends a signal to the LLM indicating precisely which n-gram (leveraging SpeakStream's word-level alignment) was being spoken at the interruption point. This allows the LLM to selectively clear its key-value cache of content that was generated but not yet vocalized, maintaining conversational coherence across turns (see Figure 1). The TTS and vocoder components do not require cache clearing as they reinitialize the cache for each turn. Following interruption, all system components transition to a new conversation turn state, ready to process the user's input.

The interruption sensitivity can be tuned via a configurable parameter that controls the pause duration on which we invoke interruptions. Due to continuous listening by the ASR, interruption and its feedback mechanisms, we can set a low value for this parameter to reduce the latency of the system. Thus our system always monitors whether the user is talking or not and prioritizes user speech over agent speech: if an interruption is detected after the LLM has generated a response that has not yet been vocalized while the user continues to speak, then (a) the system discards the unvocalized generated response; (b) control will be returned to the user turn; (c) the system knows in which state the LLM should be given the feedback mechanism in interruption handling.

C. Context Management and Chunking

ChipChat implements context management strategies to handle the inherent limitations of fixed-context models while maintaining conversational coherence. For the ASR model, we use a temporal chunking that fully resets the model's key-value cache after a configurable period of silence. This approach balances the benefits of contextual information for ongoing speech with the need to avoid limited generalization of positional embedding to longer contexts. The LLM employs rotating key-value caching. By default, the system maintains context for N conversation turns (configurable) before performing a full cache reset. For the TTS model, we implement sentence-level chunking within a streaming framework. The system monitors incoming text from the LLM for sentence-final punctuation (.!?) and, upon detection, completes the current sentence generation before clearing the key-value cache. This approach ensures prosodic coherence within sentences while allowing for natural breaks between them. To minimize latency during these transitions, we employ carefully designed short prefixes that end with appropriate pauses, ensuring that the first generated frame contains phonetic content rather than silence. The vocoder, being a convolutional model with limited temporal dependencies, does not require explicit chunking strategies, allowing for continuous audio generation across sentence boundaries when appropriate.

D. System Flexibility

As a cascaded architecture, ChipChat offers flexibility by supporting multiple configurations: ASR-only, ASR and LLM, LLM-only, TTS and vocoder, LLM with TTS and vocoder, vocoder-only, or end-to-end. This modularity facilitates both research experimentation and practical deployment scenarios with varying resource constraints.

III. Related work and Conclusion

ESPnet-SDS [14] is an open-source unified web interface for various cascaded and E2E spoken dialog systems, but runs on a GPU server, and is not designed/optimized for streaming CS. Recently, Kyutai announced their streaming CS [30] although technical details have not yet been published. Our initial naive implementation of the CS with Pytorch (except the LLM which is still in MLX) and without the aforementioned optimizations had latency more than 4s.

In the paper, we introduce ChipChat, a low-latency cascaded system for conversational agents that overcomes traditional latency limitations while preserving the quality advantages of specialized components. ChipChat shows that redesigned parts of cascaded systems with a focus on overall system performance offer a compelling alternative for next-generation

conversational agents. ChipChat achieves sub-second response latency on Mac Studio (Table I), running entirely on-device.

IV. Acknowledgments

We would like to thank: Angelos Katharopoulos, Awni Hanun, Jagrit Digani, Ronan Collobert for help with MLX; Samy Bengio, Christopher Webb, Jamie Cheng for the feedback.

References

[1] J. Glass, "Challenges for spoken dialogue systems," in *Proceedings of the 1999 IEEE ASRU Workshop*, vol. 696. MIT Laboratory for Computer Science Cambridge, 1999.

[2] R. Huang, M. Li, D. Yang, J. Shi, X. Chang, Z. Ye, Y. Wu, Z. Hong, J. Huang, J. Liu *et al.*, "Audiogpt: understanding and generating speech, music, sound, and talking head," in *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, 2024, pp. 23 802–23 804.

[3] K. An, Q. Chen, C. Deng, Z. Du, C. Gao, Z. Gao, Y. Gu, T. He, H. Hu, K. Hu *et al.*, "Funaudiollm: Voice understanding and generation foundation models for natural interaction between humans and llms," *arXiv preprint arXiv:2407.04051*, 2024.

[4] T. A. Nguyen, B. Muller, B. Yu, M. R. Costa-Jussa, M. Elbayad, S. Popuri, C. Ropers, P.-A. Duquenne, R. Algayres, R. Mavlyutov *et al.*, "Spirit-lm: Interleaved spoken and written language model," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 30–52, 2025.

[5] D. Zhang, S. Li, X. Zhang, J. Zhan, P. Wang, Y. Zhou, and X. Qiu, "Speechgpt: Empowering large language models with intrinsic cross-modal conversational abilities," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 15 757–15 773.

[6] C. Tang, W. Yu, G. Sun, X. Chen, T. Tan, W. Li, L. Lu, Z. MA, and C. Zhang, "Salmonn: Towards generic hearing abilities for large language models," in *The Twelfth International Conference on Learning Representations*, 2024.

[7] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, "Moshi: a speech-text foundation model for real-time dialogue," *arXiv preprint arXiv:2410.00037*, 2024.

[8] Q. Fang, Y. Zhou, S. Guo, S. Zhang, and Y. Feng, "Llama-omni2: Llm-based real-time spoken chatbot with autoregressive streaming speech synthesis," *arXiv preprint arXiv:2505.02625*, 2025.

[9] Z. Xie and C. Wu, "Mini-omni: Language models can hear, talk while thinking in streaming," *arXiv preprint arXiv:2408.16725*, 2024.

[10] J. Xu, Z. Guo, J. He, H. Hu, T. He, S. Bai, K. Chen, J. Wang, Y. Fan, K. Dang *et al.*, "Qwen2. 5-omni technical report," *arXiv preprint arXiv:2503.20215*, 2025.

[11] S. Ji, Y. Chen, M. Fang, J. Zuo, J. Lu, H. Wang, Z. Jiang, L. Zhou, S. Liu, X. Cheng *et al.*, "Wavchat: A survey of spoken dialogue models," *arXiv preprint arXiv:2411.13577*, 2024.

[12] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.

[13] Picovoice.AI, "Text-to-speech latency benchmark," https://picovoice.ai/docs/benchmark/tts-latency/, 2024.

[14] S. Arora, Y. Peng, J. Shi, J. Tian, W. Chen, S. Bharadwaj, H. Futami, Y. Kashiwagi, E. Tsunoo, S. Shimizu *et al.*, "Espnet-sds: Unified toolkit and demo for spoken dialogue systems," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*, 2025, pp. 248–259.

[15] Y. Chen, X. Yue, C. Zhang, X. Gao, R. T. Tan, and H. Li, "Voicebench: Benchmarking llm-based voice assistants," *arXiv preprint arXiv:2410.17196*, 2024.

[16] S. Sakshi, U. Tyagi, S. Kumar, A. Seth, R. Selvakumar, O. Nieto, R. Duraiswami, S. Ghosh, and D. Manocha, "Mmau: A massive multi-task audio understanding and reasoning benchmark," in *The Thirteenth International Conference on Learning Representations*, 2025.

[17] S. Arora, Z. Lu, C.-C. Chiu, R. Pang, and S. Watanabe, "Talking turns: Benchmarking audio foundation models on turn-taking dynamics," in *The Thirteenth International Conference on Learning Representations*, 2025.

[18] P. Mousavi, G. Maimon, A. Moumen, D. Petermann, J. Shi, H. Wu, H. Yang, A. Kuznetsova, A. Ploujnikov, R. Marxer *et al.*, "Discrete audio tokens: More than a survey!" *arXiv preprint arXiv:2506.10274*, 2025.

[19] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert, "MLX: Efficient and flexible machine learning on apple silicon," 2023. [Online]. Available: https://github.com/ml-explore

[20] "Rabbitmq documentation," https://www.rabbitmq.com/docs.

[21] Z. Gu, T. Likhomanenko, and N. Jaitly, "Omni-router: Sharing routing decisions in sparse mixture-of-experts for speech recognition," *arXiv preprint arXiv:2507.05724*, 2025.

[22] A. Graves, "Connectionist temporal classification," in *Supervised sequence labelling with recurrent neural networks*. Springer, 2012, pp. 61–93.

[23] Z. Aldeneh, T. Higuchi, J.-w. Jung, L.-W. Chen, S. Shum, A. H. Abdelaziz, S. Watanabe, T. Likhomanenko, and B.-J. Theobald, "Speaker-ipl: Unsupervised learning of speaker characteristics with i-vector based pseudo-labels," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.

[24] Y. Zhang and N. Jaitly, "Sage: Steering and refining dialog generation with state-action augmentation," *arXiv preprint arXiv:2503.03040*, 2025.

[25] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[26] "Mlx lm," https://github.com/ml-explore/mlx-lm.

[27] R. H. Bai, Z. Gu, T. Likhomanenko, and N. Jaitly, "Speakstream: Streaming text-to-speech with interleaved data," *arXiv preprint arXiv:2505.19206*, 2025.

[28] R. H. Bai, T. Likhomanenko, R. Zhang, Z. Gu, Z. Aldeneh, and N. Jaitly, "dmel: Speech tokenization made simple," *arXiv preprint arXiv:2407.15835*, 2024.

[29] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, "Gradio: Hassle-free sharing and testing of ml models in the wild," *arXiv preprint arXiv:1906.02569*, 2019.

[30] Kyutai, "Kyutai unmute," https://unmute.sh/, 2025.